



འབྲུག་རྒྱལ་ཁཛུགས་གཞི་རྒྱུ་

College of Science and Technology
Rinchending: Bhutan

WEB101

Web Application Fundamentals (SS2026)

Practical 3 Report

Submitted By;

Name: Kinley Pem

Student No.: 02250354

Programme: BE 1SWE

Date: 16/04/2026

RUB Wheel of Academic Law: Academic Dishonesty

Section H2 of the Royal University of Bhutan's *Wheel of Academic Law* provides the following definition of academic dishonesty:

Academic dishonesty may be defined as any attempt by a student to gain an unfair advantage in any assessment. It may be demonstrated by one of the following:

1. **Collusion:** the representation of a piece of unauthorised group work as the work of a single candidate.
2. **Commissioning:** submitting an assignment done by another person as the student's own work.
3. **Duplication:** the inclusion in coursework of material identical or substantially similar to material which has already been submitted for any other assessment within the University.
4. **False declaration:** making a false declaration to receive special consideration by an Examination Board or to obtain extensions to deadlines or exemption from work.
5. **Falsification of data:** presentation of data in laboratory reports, projects, etc., based on work purported to have been carried out by the student, which has been invented, altered or copied by the student.
6. **Plagiarism:** the unacknowledged use of another's work as if it were one's own.

Examples are:

- verbatim copying of another's work without acknowledgement.
- paraphrasing of another's work by simply changing a few words or altering the order of presentation, without acknowledgement.
- ideas or intellectual data in any form presented as one's own without acknowledging the source(s).
- making significant use of unattributed digital images such as graphs, tables, photographs, etc., taken from test books, articles, films, plays, handouts, the internet, or any other source, whether published or unpublished.
- submission of a piece of work which has previously been assessed for a different award or module or at a different institution as if it were new work.
- use of any material without prior permission of copyright from the appropriate authority or owner of the materials used".

Aim

The primary aim of this practical is to learn to:

- Build a file upload application using Next.js that demonstrates:
 - i. Creating a file upload form with drag-and-drop functionality
 - ii. Implementing client-side file validation (size and type restrictions)
 - iii. Building a server-side API route to handle file uploads
 - iv. Using React Hook Form for form management
 - v. Implementing upload progress tracking
 - vi. Displaying file previews for images and PDFs

Objectives

- To create a Next.js project with proper folder structure
- To install required dependencies (react-hook-form, formidable, axios, react-dropzone)
- To build an HTML form with name input and file upload field
- To implement drag-and-drop file upload using react-dropzone
- To add client-side validation for file size (max 5MB) and file type (JPEG, PNG, PDF)
- To create an API route in Next.js for handling file uploads
- To implement file preview functionality for images and PDFs
- To display upload progress and success/error messages
- To save uploaded files to the server's public/uploads directory

Theory

In this practical, I learned how to build a file upload application using Next.js, which is a React framework for building web applications.

What is Next.js?

Next.js is a React framework that provides features like server-side rendering, API routes, and file-based routing. It makes building full-stack React applications easier by handling many common configurations and optimizations.

What are API Routes in Next.js?

API routes in Next.js allow us to create backend endpoints within the same project. These routes are stored in the `app/api/` directory and can handle HTTP requests like POST, GET, PUT, and DELETE. For this practical, I created a POST endpoint at `/api/upload` to handle file uploads.

React Hook Form

React Hook Form is a library that helps manage form state and validation in React applications. It reduces the amount of code needed and improves performance by using uncontrolled components.

React Dropzone

React Dropzone is a library that provides drag-and-drop file upload functionality. It handles the complex logic of detecting drag events, file rejection, and providing accessibility features.

File Validation

File validation is crucial for security and user experience. In this application, I implemented:

Validation Rule	Purpose
File size $\leq$ 5MB	Prevents excessively large files from being uploaded
File type (JPEG, PNG, PDF)	Ensures only allowed file formats are accepted
Name field required	Ensures user provides their name

Axios

Axios is a promise based HTTP client that works in both the browser and Node.js. It provides features like request/response interceptors, progress tracking, and automatic JSON transformation.

How the File Upload App Works

1. User enters their name in the text field
2. User selects a file by clicking the dropzone or dragging a file
3. Client-side validation checks file size and type
4. Preview of the file is displayed (image preview or PDF/file name)
5. User clicks the "Upload File" button
6. Form data is sent to the /api/upload endpoint using Axios
7. Server saves the file to the public/uploads directory with a unique filename
8. Upload progress is displayed during the upload process
9. Success or error message is shown to the user

PROCEDURE

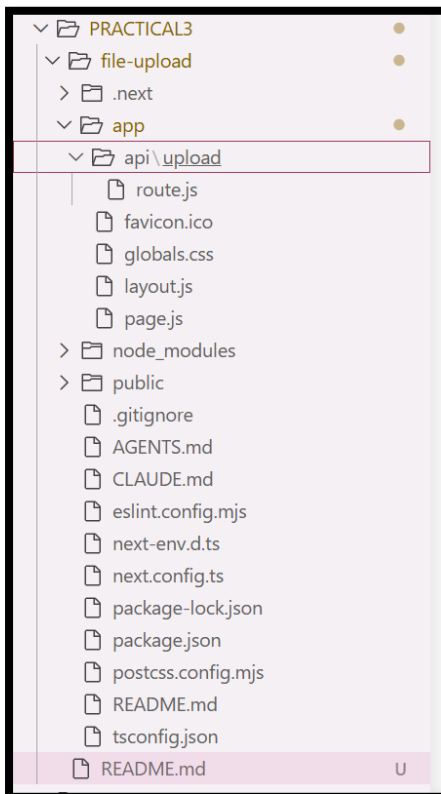
Part A: Project Setup

Step 1: Create a New Next.js Project

First, I created a new Next.js project using the create-next-app command:

```
npx create-next-app file-upload  
cd file-upload
```

Project Structure Created:



Step 2: Install Required Dependencies

Next, I installed the necessary packages for the file upload functionality:

```
npm install react-hook-form formidable axios react-dropzone
```

Dependencies Installation:

```
Success! Created file-upload at D:\2nd sem\WEB101 PRACTICALS\PRACTICAL3\file-upload  
PS D:\2nd sem\WEB101 PRACTICALS\PRACTICAL3> cd file-upload  
PS D:\2nd sem\WEB101 PRACTICALS\PRACTICAL3\file-upload> npm install react-hook-form formidable axios react-dropzone  
added 20 packages, and audited 376 packages in 14s
```

Step 3: Verify Project Running

After installation, I started the development server to verify everything was working:

```
npm run dev
```

Development Server Running:

```
PS D:\2nd sem\WEB101 PRACTICALS\PRACTICAL3> cd file-upload
PS D:\2nd sem\WEB101 PRACTICALS\PRACTICAL3\file-upload> npm run dev

> file-upload@0.1.0 dev
> next dev --turbo

▲ Next.js 16.2.3 (Turbopack)
- Local:      http://localhost:3000
- Network:    http://10.2.30.83:3000
✓ Ready in 814ms

GET / 200 in 1541ms (next.js: 727ms, application-code: 814ms)
POST /api/upload 200 in 311ms (next.js: 218ms, application-code: 93ms)
```

Part B: Creating the User Interface

Step 4: Configure Global CSS

I updated the globals.css file to include custom styles for the dropzone, progress bar, file list, and preview container.

Global CSS:

```
PRACTICAL3 > file-upload > app > globals.css > ...
1 @import "tailwindcss";
2
3 :root {
4   --background: #ffffff;
5   --foreground: #171717;
6 }
7
8 :root {
9   --color-background: var(--background);
10  --color-foreground: var(--foreground);
11  --font-sans: var(--font-geist-sans);
12  --font-mono: var(--font-geist-mono);
13 }
14
15 @media (prefers-color-scheme: dark) {
16   :root {
17     --background: #0a0a0a;
18     --foreground: #ededed;
19   }
20 }
```

```

21
22 body {
23   background: var(--background);
24   color: var(--foreground);
25   font-family: Arial, Helvetica, sans-serif;
26 }
27
28 /* File Upload Styles */
29 .dropzone {
30   transition: all 0.3s ease;
31 }
32

```

```

33 .dropzone.active {
34   border-color: #3b82f6;
35   background-color: #fff6ff;
36 }
37
38 .progress-container {
39   width: 100%;
40   height: 20px;
41   background-color: #e5e7eb;
42   border-radius: 9999px;
43   margin: 1rem 0;
44   overflow: hidden;
45 }
46
47 .progress-bar {
48   height: 100%;
49   background-color: #3b82f6;
50   transition: width 0.3s ease;
51 }
52
53 /* File list styles */
54 .file-list {
55   margin-top: 1rem;
56 }
57
58 .file-list ul {
59   list-style: none;
60   padding: 0;
61 }
62
63 .file-list li {
64   display: flex;
65   align-items: center;
66   justify-content: space-between;
67   padding: 0.5rem;
68   margin-bottom: 0.5rem;
69   background-color: #f3f4f6;
70   border-radius: 0.25rem;
71 }

```

```
72
73 .remove-btn {
74   background-color: #ef4444;
75   color: white;
76   border: none;
77   border-radius: 9999px;
78   width: 1.5rem;
79   height: 1.5rem;
80   display: flex;
81   align-items: center;
82   justify-content: center;
83   cursor: pointer;
84 }
85
86 /* Preview image styles */
87 .preview-container {
88   margin-bottom: 1rem;
89   border: 1px solid #e5e7eb;
90   border-radius: 0.25rem;
91   padding: 0.5rem;
92 }
93
94 .preview-image {
95   max-width: 100%;
96   max-height: 200px;
97   object-fit: contain;
98 }
99
100 .preview-info {
101   margin-top: 0.5rem;
102   font-size: 0.875rem;
103   color: #6b7280;
104 }
```

Step 5: Configure Layout

The layout file sets up the HTML structure, font imports, and metadata for the application.

Layout Configuration:

```
PRACTICAL3 > file-upload > app > layout.js > geistMono

1 import { Geist, Geist_Mono } from "next/font/google";
2 import "../globals.css";
3
4 const geistSans = Geist({
5   variable: "--font-geist-sans",
6   subsets: ["latin"],
7 });
8
9 const geistMono = Geist_Mono({
10  variable: "--font-geist-mono",
11  subsets: ["latin"],
12 });
13
14 export const metadata = {
15   title: "Create Next App",
16   description: "Generated by create next app",
17 };
18
19 export default function RootLayout({ children }) {
20   return (
21     <html lang="en">
22       <body
23         className={` ${geistSans.variable} ${geistMono.variable} antialiased`>
24         >
25         {children}
26       </body>
27     </html>
28   );
29 }
30
```

Step 6: Create the Main Page Component

I created the main page component (page.js) with the following sections:

1. **State Management** - Using useState hooks for upload progress, upload status, and file preview
2. **Form Setup** - Using React Hook Form with validation rules
3. **Dropzone Component** - Custom drag-and-drop file upload component
4. **Form UI** - Name input field and file upload area
5. **Preview Section** - Display image preview or PDF information
6. **Progress Tracking** - Show upload progress percentage
7. **Result Display** - Show success or error message

Page.js:

```
PRACTICAL3 > file-upload > app > page.js > FileUploadForm > Dropzone

1 "use client"
2
3 import { useState, useCallback } from 'react';
4 import { useForm, Controller } from 'react-hook-form';
5 import axios from 'axios';
6 import { useDropzone } from 'react-dropzone';
7
8 export default function FileUploadForm() {
9   // Define constants for validation
10  const MAX_FILE_SIZE = 5 * 1024 * 1024; // 5MB Limit
11  const ACCEPTED_FILE_TYPES = ['image/jpeg', 'image/png', 'application/pdf'];
12
13  // Initialize React Hook Form
14  const { control, register, handleSubmit, formState: { errors } } = useForm();
15
16  // State for tracking upload progress and status
17  const [uploadProgress, setUploadProgress] = useState(0);
18  const [isUploading, setIsUploading] = useState(false);
19  const [uploadResult, setUploadResult] = useState(null);
20  const [filePreview, setFilePreview] = useState(null);
21
22  // Update the onSubmit function to use Next.js API route
23  const onSubmit = async (data) => {
24    setIsUploading(true);
25    setUploadProgress(0);
26    setUploadResult(null);
27
28    try {
29      const formData = new FormData();
30      formData.append('file', data.file[0]);
31      formData.append('name', data.name);
32
33      // Log what we're trying to upload for debugging
34      console.log('Uploading file:', data.file[0].name, 'Type:', data.file[0].type);
35
36      // CHANGE THIS LINE - Use Next.js API route instead of Express backend
37      const response = await axios.post('/api/upload', formData, {
38        onUploadProgress: (progressEvent) => {
39          const percentage = Math.round(
40            (progressEvent.loaded * 100) / progressEvent.total
41          );
42          setUploadProgress(percentage);
43        }
44      });
45
46      setUploadResult({
47        success: true,
48        message: 'File uploaded successfully!',
49        data: response.data
50      });
51    } catch (error) {
52      console.error('Upload error:', error);
53      setUploadResult({
54        success: false,
55        message: error.response?.data?.error || 'Upload failed'
56      });
57    } finally {
58      setIsUploading(false);
59    }
60  };
61 }
```

```

63 // Custom Dropzone component for drag & drop functionality
64 const Dropzone = ({ onDrop, maxSize, accept }) => {
65   // Initialize react-dropzone with configuration
66   const {
67     getRootProps,
68     getInputProps,
69     isDragActive,
70     fileRejections
71   } = useDropzone({
72     onDrop: (acceptedFiles) => {
73       // Process the dropped files
74       onDrop(acceptedFiles);
75
76       // Create preview info for the file (both images and PDFs)
77       if (acceptedFiles.length > 0) {
78         const file = acceptedFiles[0];
79
80         // For images, create preview URL
81         if (file.type.startsWith('image/')) {
82           const previewUrl = URL.createObjectURL(file);
83           setFilePreview({
84             url: previewUrl,
85             name: file.name,
86             type: file.type
87           });
88         }
89         // For PDFs, just store the name and type (no preview URL needed)
90         else if (file.type === 'application/pdf') {
91           setFilePreview({
92             name: file.name,
93             type: file.type
94           });
95         }
96         // For other file types
97         else {
98           setFilePreview(null);
99         }
100       }
101     },
102     maxSize,
103     accept,
104     multiple: false // Allow only single file upload
105   });
106
107   // Display rejected files with reasons
108   const fileRejectionItems = fileRejections.map(({ file, errors }) => (
109     <li key={file.path} className="text-red-500">
110       {file.name} - {errors.map(e => e.message).join(', ')}
111     </li>
112   ));
113
114   // Render the dropzone UI
115   return (
116     <div className="mt-1">
117       <div>
118         { /* Dropzone container with styling based on drag state */ }
119         <div>
120           { ...getRootProps() }
121           className={
122             {
123               'border-2 border-dashed rounded-lg p-6 cursor-pointer text-center':
124                 isDragActive ? 'border-blue-500 bg-blue-50' : 'border-gray-300 hover:border-blue-400'
125             }
126           }
127         </div>
128         { /* Hidden file input */ }
129         <input {...getInputProps()} />
130
131         { /* Display different message based on drag state */ }
132         { isDragActive ? (
133           <p className="text-blue-500">Drop the file here...</p>
134         ) : (
135           <div>
136             <p>Drag & drop a file here, or click to select</p>
137             <p className="text-sm text-gray-500 mt-1">
138               (Only JPEG, PNG, and PDF files under 5MB are accepted)
139             </p>
140             <input type="text" value="" />
141           </div>
142         ) }
143       </div>
144
145       { /* Display any file validation errors */ }
146       { fileRejectionItems.length > 0 && (
147         <ul className="mt-2">{fileRejectionItems}</ul>
148       ) }
149     </div>
150   );
151
152   // Main component render
153   return (
154     <div className="container mx-auto p-4">
155       <h1 className="text-2xl font-bold mb-6">File Upload Form</h1>
156
157       { /* Form with validation */ }
158       <form onSubmit={handleSubmit(onSubmit)} className="max-w-md">
159         { /* Name input field */ }
160         <div className="mb-4">
161           <label htmlFor="name" className="block mb-1 font-medium">Your Name:</label>
162           <input type="text" value="" />
163         </div>
164       </form>
165     </div>
166   );
167 }
168
169 export default Dropzone;

```

```

158     id="name"
159     type="text"
160     {...register('name', { required: 'Name is required' })}
161     className="w-full p-2 border rounded"
162   />
163   { /* Display name field validation errors */ }
164   {errors.name && <p className="text-red-500 mt-1">{errors.name.message}</p>}
165 </div>
166
167 { /* File upload field with Controller for custom input handling */ }
168 <div className="mb-4">
169   <label className="block mb-1 font-medium">Upload File:</label>
170   <Controller
171     name="file"
172     control={control}
173     rules={{
174       required: 'File is required',
175       validate: {
176         // Validate file size
177         fileSize: files => !files?.[0] || files[0].size <= MAX_FILE_SIZE ||
178         | 'File size must be less than 5MB',
179         // Validate file type
180         fileType: files => !files?.[0] || ACCEPTED_FILE_TYPES.includes(files[0].type) ||
181         | 'Only JPEG, PNG, and PDF files are accepted'
182       }
183     }}
184     render={({ field: { onChange, value } }) => (
185       <Dropzone
186         onDrop={acceptedFiles => onChange(acceptedFiles)}
187         maxSize={MAX_FILE_SIZE}
188         accept={{
189           'image/jpeg': ['.jpg', '.jpeg'],
190           'image/png': ['.png'],
191           'application/pdf': ['.pdf']
192         }}
193       />
194     )
195   />
196   { /* Display file field validation errors */ }
197   {errors.file && <p className="text-red-500 mt-1">{errors.file.message}</p>}
198 </div>
199
200 { /* Preview section - show for both images and PDFs */ }
201 {filePreview && (
202   <div className="mb-4">
203     <h3 className="font-medium mb-1">Preview:</h3>
204     <div className="border rounded p-2">
205       {filePreview.type?.startsWith('image/') ? (
206         /* Show actual preview for images */
207         <img
208           src={filePreview.url}
209           alt={filePreview.name}
210           className="max-w-full h-auto max-h-40 rounded"
211         />
212       ) : filePreview.type === 'application/pdf' ? (
213         /* Just show file name for PDFs */
214         <div className="py-2 px-3 bg-gray-100 rounded flex items-center">
215           <svg className="w-6 h-6 text-red-500 mr-2" fill="currentColor" viewBox="0 0 20 20">
216             <path d="M9 2a2 2 0 0-2 2v8a2 2 0 02 2h6a2 2 0 02-2V6.414A2 2 0 016.414 5L14 2.586A2 2 0 012.586 2H9z"></path>
217             <path d="M3 8a2 2 0 012-2h2a2 2 0 012 2v8a2 2 0 01-2 2H5a2 2 0 01-2-2V8z"></path>
218           </svg>
219           <span>{filePreview.name}</span>
220         </div>
221       ) : (
222         /* Generic file info for other types */
223         <div>File selected: {filePreview.name}</div>
224       )
225     }
226   </div>
227 )}
228
229 { /* Submit button with loading state */ }
230 <button
231   type="submit"
232   disabled={isLoading}
233   className={`w-full p-2 text-white rounded ${
234     isLoading ? 'bg-blue-400' : 'bg-blue-500 hover:bg-blue-600'
235   }`}
236 >
237   {isLoading ? 'Uploading...' : 'Upload File'}
238 </button>

```

```

239     /* Progress bar */
240     {isUploading && (
241         <div className="mt-4">
242             <div className="w-full bg-gray-200 rounded-full h-2.5">
243                 <div
244                     className="bg-blue-600 h-2.5 rounded-full transition-all duration-300"
245                     style={{ width: `${uploadProgress}%` }}
246                 ></div>
247             </div>
248             <p className="text-center mt-1 text-sm">{uploadProgress}%</p>
249         </div>
250     )}
251 }
252
253     /* Upload result message */
254     {uploadResult && (
255         <div className={`mt-4 p-3 rounded ${
256             uploadResult.success ? 'bg-green-100 text-green-800' : 'bg-red-100 text-red-800'
257         }`}>
258             <p>{uploadResult.message}</p>
259             {uploadResult.success && uploadResult.data?.filename && (
260                 <p className="text-sm mt-1">Uploaded as: {uploadResult.data.filename}</p>
261             )}
262         </div>
263     )}
264 </form>
265 </div>
266 );
267 }

```

Part C: Creating the API Route

Step 7: Create the Upload API Route

I created an API route at `app/api/upload/route.js` to handle file uploads on the server side.

API Route Implementation:

```
PRACTICAL3 > file-upload > app > api > upload > route.js > POST

1 import { writeFile, mkdir } from 'fs/promises';
2 import { NextResponse } from 'next/server';
3 import { join } from 'path';
4
5 export async function POST(request) {
6   try {
7     const formData = await request.formData();
8     const file = formData.get('file');
9
10    if (!file) {
11      return NextResponse.json(
12        { error: 'No file uploaded' },
13        { status: 400 }
14      );
15    }
16
17    const bytes = await file.arrayBuffer();
18    const buffer = Buffer.from(bytes);
19
20    // Create the uploads directory if it doesn't exist
21    const uploadDir = join(process.cwd(), 'public', 'uploads');
22    await mkdir(uploadDir, { recursive: true });
23
24    // Create a unique filename
25    const uniqueFilename = `${Date.now()}-${file.name}`;
26    const filePath = join(uploadDir, uniqueFilename);
27
28    // Save the file
29    await writeFile(filePath, buffer);
30
31    return NextResponse.json({
32      message: 'File uploaded successfully',
33      filename: uniqueFilename,
34      originalName: file.name,
35      url: `~/uploads/${uniqueFilename}`,
36    });
37  } catch (error) {
38    console.error('Error uploading file:', error);
39    return NextResponse.json(
40      { error: `Error uploading file: ${error.message}` },
41      { status: 500 }
42    );
43  }
44 }
45 }
```

The API route performs the following operations:

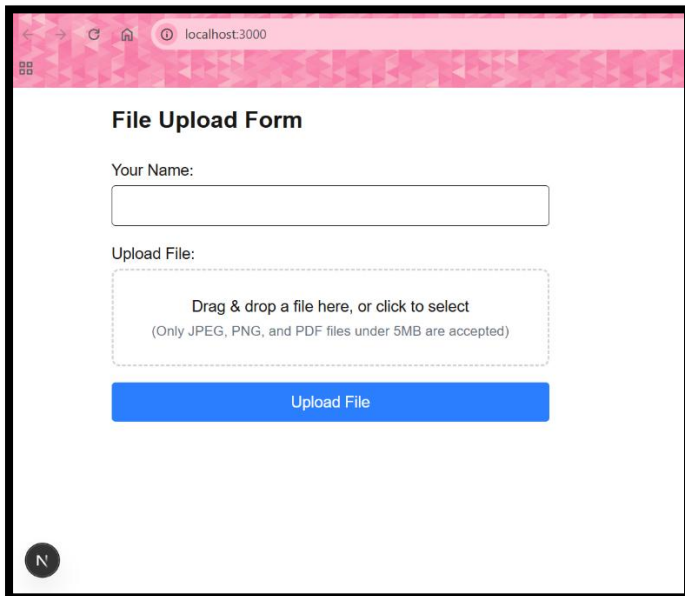
- i. Extracts the file from the form data
- ii. Converts the file to a buffer
- iii. Creates the uploads directory if it doesn't exist
- iv. Generates a unique filename using timestamp
- v. Saves the file to `public/uploads/`
- vi. Returns the file information in the response

Part D: Testing the Application

Step 8: Initial Form Display

When the application is first loaded, the user sees the file upload form with the name field and drag-and-drop area.

Initial Form Display:

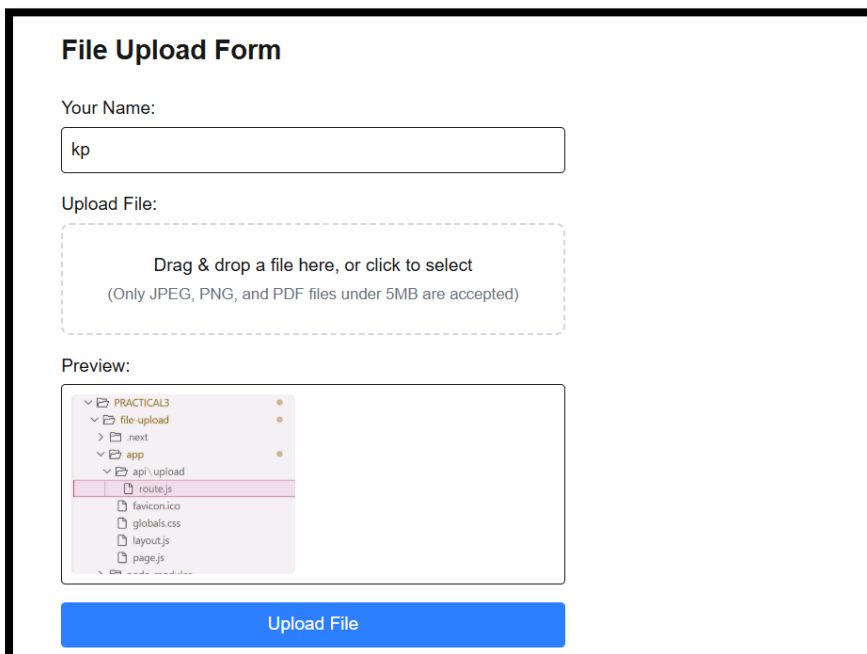


A screenshot of a web browser showing a file upload form. The browser's address bar displays 'localhost:3000'. The form is titled 'File Upload Form' and contains a text input field labeled 'Your Name:'. Below this is a dashed box for file upload with the text 'Drag & drop a file here, or click to select' and '(Only JPEG, PNG, and PDF files under 5MB are accepted)'. A blue 'Upload File' button is at the bottom. A small circular icon with a 'Z' is in the bottom left corner.

Step 9: Enter Name and Select File

The user enters their name and selects a file by clicking the dropzone or dragging a file.

Form with Data and Preview:

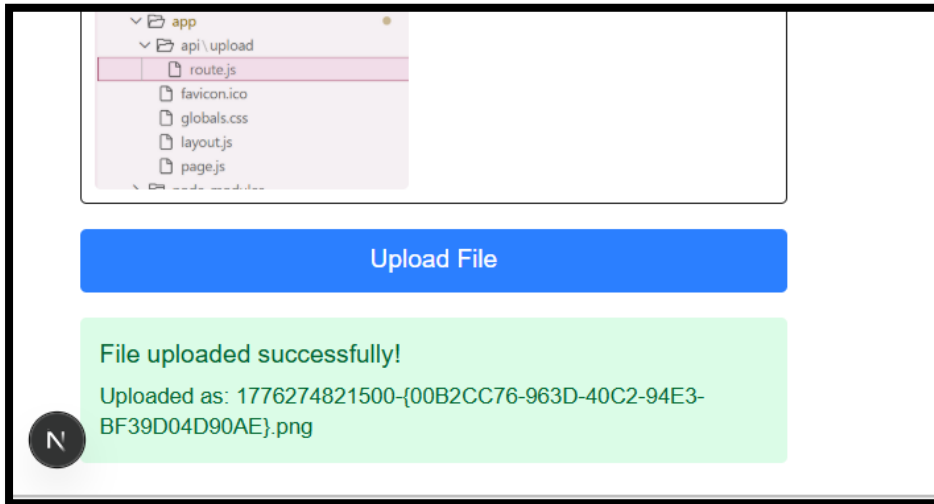


A screenshot of the same file upload form, now with user input. The 'Your Name:' field contains the text 'kp'. The file upload area remains the same. Below it, a 'Preview:' section shows a file explorer view with a tree structure. The tree includes folders like 'PRACTICAL3', 'file upload', '.next', 'app', and 'api \ upload', and files like 'route.js', 'favicon.ico', 'globals.css', 'layout.js', and 'page.js'. The 'route.js' file is selected. A blue 'Upload File' button is at the bottom.

Step 10: File Upload Success

After clicking the "Upload File" button, the file is uploaded to the server, and a success message is displayed with the uploaded filename.

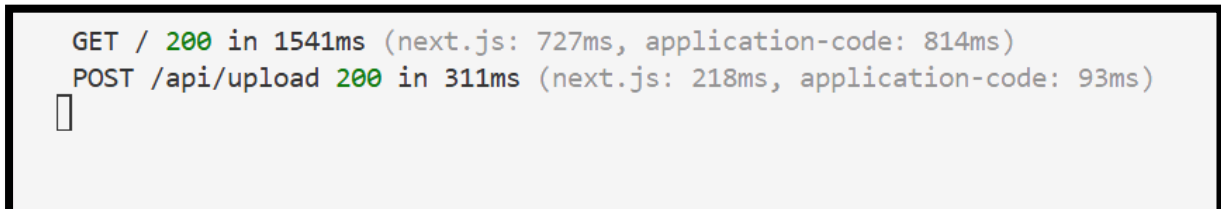
Upload Success Message:



Step 11: Server Logs

The terminal shows the server receiving and processing the upload request.

Server Logs:



CONCLUSION

In this practical, I successfully built a file upload application using Next.js that demonstrates both client-side and server-side functionality. I managed to complete all the objectives:

- i. **Project Setup:** Created a Next.js project and installed all required dependencies
- ii. **Form Implementation:** Built a form with name input and file upload using React Hook Form
- iii. **Drag-and-Drop:** Implemented drag-and-drop functionality using react-dropzone
- iv. **Client Validation:** Added file size (5MB limit) and file type (JPEG, PNG, PDF) validation
- v. **Server API:** Created a Next.js API route at /api/upload to handle file uploads
- vi. **File Storage:** Implemented server-side file saving with unique filenames
- vii. **Preview Functionality:** Displayed image previews for images and file info for PDFs
- viii. **Progress Tracking:** Showed upload progress using Axios progress events
- ix. **User Feedback:** Displayed success/error messages after upload completion

REFERENCES

1. College of Science and Technology Rinchending. (2026). *WEB101 Practical 3: File upload implementation* [PDF document]. Bhutan: College of Science and Technology.
2. JSONPlaceholder. (n.d.). *JSONPlaceholder: Free fake API for testing and prototyping*. Retrieved April 15, 2026, from <https://jsonplaceholder.typicode.com/>
3. MDN Web Docs. (2025). *Using the Fetch API*. Retrieved April 15, 2026, from https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch
4. Mozilla Developer Network. (n.d.). *HTTP request methods*. Retrieved April 15, 2026, from <https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods>
5. OpenWeatherMap. (n.d.). *Current weather data*. Retrieved April 15, 2026, from <https://openweathermap.org/current>
6. Syangche. (n.d.). *React_Practicals* [GitHub repository]. Retrieved April 15, 2026, from https://github.com/syangche/React_Practicals.git